

Gonka Message House

Version: 1.0 **Date:** 2026-04-01 **Classification:** Internal – canonical messaging framework for all Gonka developer-facing communications **Feeds into:** Phase 18 (Channel Strategy), Phase 19 (Partnership & Ecosystem) **Requirement:** MSG-02

1. Executive Summary

This document is the single source of truth for how Gonka talks to developers. Every blog post, landing page, documentation snippet, conference talk, comparison table, and community reply should reference this framework before publishing. It contains: a core positioning statement that leads with developer outcomes; ranked value propositions grounded in competitive evidence; per-persona differentiation for three developer archetypes; a complete architecture-to-message mapping that translates every Gonka technical feature into a developer benefit; vocabulary guidelines that prevent crypto jargon from leaking into developer-facing content; and competitive differentiation statements tailored to the competitor each persona is most likely comparing Gonka against. All claims are sourced from Phase 15 (competitive analysis) and Phase 16 (developer personas) research, with parenthetical citations throughout.

2. Core Positioning Statement

For OpenClaw developers who spend too much on inference because their agents resend full context on every heartbeat, Gonka is the agent-native inference provider that cuts agent costs by up to 73% through server-side session persistence, unlike OpenRouter and Together AI which charge for every token of repeated context, because Gonka maintains conversation state server-side so your agent never pays for the same context twice.

Positioning Statement Breakdown

Element	Content	Source
Target audience	OpenClaw developers running always-on AI agents	(developer personas: all three personas use OpenClaw)
Need	Agents resend full context on every heartbeat, inflating costs by 44-85%	(pricing analysis: Section 3, heartbeat overhead by tier)
Category	Agent-native inference provider	(provider landscape map: Gonka is the only provider combining decentralized infrastructure with agent-native extensions)
Key benefit	Cuts agent costs by up to 73% through server-side session persistence	(pricing analysis: Section 6, Active tier DeepInfra \$218/mo vs Gonka Scenario B \$59/mo)
Alternatives	OpenRouter and Together AI	(competitive matrix: both scored LOSE on Agent Sessions)
Reason to believe	Server-side conversation state eliminates repeated context transmission	(competitive matrix: Agent Sessions – WIN; architecture mapping: X-Gonka-Session-ID)

Why This Positioning Works

- Leads with developer outcome** (cost reduction), not technology (decentralization) – per CONTEXT.md locked decision and PITFALLS.md Pitfall 1
- Names a specific, quantified pain** (heartbeat overhead) that developers experience personally – per pricing analysis Section 3
- Names the specific mechanism** (session persistence) rather than a vague “decentralized compute” claim – per PITFALLS.md Anti-Pattern 2
- Names specific alternatives** the audience is already using, not generic “other providers” – per ARCHITECTURE.md Anti-Pattern 3

3. Value Propositions (Ranked)

Value propositions are ranked by differentiation strength, following the USP ranking from ARCHITECTURE.md Section “Unique Selling Propositions (Ranked)” but rewritten as developer-facing benefit statements.

VP1: Your Agent Stops Paying for Context It Already Has

Headline: Stop paying for the same tokens twice – Gonka remembers your agent’s conversation server-side.

What this means for developers: OpenClaw agents resend their full system prompt and workspace context (~9,600 tokens) on every heartbeat – 48 times per day at default 30-minute intervals. That is 460,800 tokens per day that generate zero user-facing value (pricing analysis: Section 3, Casual tier). Gonka’s server-side session persistence via `/v1/sessions` API maintains conversation state across requests, reducing heartbeat token overhead by approximately 80% (pricing analysis: Section 5, Gonka hidden cost analysis). No other provider on the standard chat completions API surface offers this capability.

Supporting evidence: - Competitive matrix verdict: Agent Sessions – **WIN** (competitive matrix: the only provider with server-side session persistence on the chat completions API) - Cost impact: Active tier drops from \$218/month (DeepInfra) to \$59/month (Gonka Scenario B) – a 73% reduction (pricing analysis: Section 6) - Casual tier: \$47/month (Together AI) drops to ~\$13/month (Gonka Scenario B) – a 72% reduction (pricing analysis: Section 5)

Resonates most with: - **Weekend Builder** – cost per task is their #1 decision driver; heartbeat waste is their primary pain point (developer personas: Weekend Builder, Decision Driver #1) - **Startup CTO** – multi-agent heartbeat overhead scales linearly with channel count; 6 channel-agents = 288 heartbeats/day (developer personas: Startup CTO, Pain Points)

Proof point: At the Active tier (6 channel-agents, 200 messages/day), heartbeats consume 51% of all tokens. Gonka’s sessions reduce this to approximately 10%, saving 2.2M tokens per day (pricing analysis: Section 3, Active tier observation).

VP2: The Right Model for Every Task – Automatically

Headline: Classification on the cheap model, complex reasoning on the strong one – your agent picks the tier, not you.

What this means for developers: OpenClaw agents perform diverse tasks within a single workflow: classifying inputs (cheap), planning approaches (moderate), executing complex reasoning (expensive). Without automatic tiering, developers either overpay (using the full model for classification) or build custom routing middleware. Gonka’s **X-Gonka-Tier** header and content-aware auto-routing (ARCHITECTURE.md: mapping table) let agents select cost-performance tiers per request – lite, mid, or full K2.5 quantization – without application code changes.

Supporting evidence: - Competitive matrix verdict: Model Tiering / Auto-Routing – **WIN** (competitive matrix: no other provider offers infrastructure-level automatic model tiering) - All four competitors scored **LOSE** on this dimension (competitive matrix: Section 2) - 76% of teams use multiple models, indicating demand for task-specific routing (developer personas: citing LangChain State of Agent Engineering)

Resonates most with: - **Startup CTO** – currently maintains custom routing middleware for model selection; Gonka eliminates that engineering overhead (developer personas: Startup CTO, Pain Point #2) - **Weekend Builder** – benefits from automatic optimization without needing to understand model routing (developer personas: Weekend Builder, Decision Driver #2, ease of integration)

Proof point: Gonka’s 3-tier routing (lite/mid/full K2.5 quantization) enables agents to use a heavily quantized model for classification at a fraction of full-model cost, then route complex reasoning to the full model – all via a single header or automatic content analysis (ARCHITECTURE.md: architecture-to-message mapping, X-Gonka-Tier).

VP3: One URL Change, Everything Else Just Works

Headline: Add Gonka to your OpenClaw agent in 90 seconds – change the base URL, keep your code.

What this means for developers: Gonka’s OpenAI-compatible API means switching from OpenRouter or Together AI requires editing one JSON block in `openclaw.json` – setting `baseUrl`, `apiKey`, and `api`: "openai-completions". No

SDK changes, no new libraries, no code modifications. The same `/v1/chat/completions` endpoint, the same streaming format, the same function calling syntax. Every OpenClaw integration test passes against Gonka's API (ARCHITECTURE.md: architecture-to-message mapping, OpenAI-compatible API).

Supporting evidence: - Competitive matrix verdict: Streaming – **TIE**; Tool Calling – **TIE** (competitive matrix: Gonka matches the standard that OpenClaw expects) - K2.5 supports 200-300 sequential tool calls in testing (competitive matrix: Section 3, verified against v1.2 integration tests) - OpenClaw's `api: "openai-completions"` config pattern makes Gonka a drop-in custom provider (ARCHITECTURE.md: OpenClaw Configuration Pattern)

Resonates most with: - **Weekend Builder** – ease of integration is their #2 decision driver; any friction beyond copy-pasting a config snippet is a dealbreaker (developer personas: Weekend Builder, Decision Driver #2) - **Startup CTO** – needs to evaluate without committing engineering resources to a provider migration (developer personas: Startup CTO, Activation stage) - **Privacy-First Builder** – evaluates by testing prompts that other providers would block; drop-in compatibility means instant testing (developer personas: Privacy-First Builder, Activation stage)

Proof point: Three fields in `openclaw.json` is all it takes: `baseUrl: "https://api.gonka.ai/v1", apiKey: "${GONKA_API_KEY}", api: "openai-completions"`. Verified working with OpenClaw, CrewAI, and LangGraph integration test suites (ARCHITECTURE.md: Component 6, OpenClaw Configuration Pattern).

VP4: No Content Filters, No Prompt Logs, No Permission Required

Headline: Your agent processes every prompt without filtering, refusal, or centralized logging.

What this means for developers: OpenAI and Anthropic enforce content policies that block legitimate use cases – legal adversarial arguments, security exploit analyses, medical condition discussions, politically sensitive content. Agents processing these tasks hit refusals that break multi-step workflows. Gonka serves K2.5, an open-weight model with no built-in content filtering, on decentralized infrastructure with no central authority that can impose content policies or review prompts (competitive matrix: content filtering comparison).

Supporting evidence: - K2.5 is open-weight with no content filtering (developer personas: Privacy-First Builder, Gonka Value Proposition) - Decentralized infrastructure means no single entity aggregates all prompt logs (developer personas: Privacy-First Builder, Gonka Value Proposition) - Privacy-First Builder persona exists specifically because content filtering on OpenAI/Anthropic blocks legitimate use cases (developer personas: Privacy-First Builder, Pain Point #1)

Resonates most with: - **Privacy-First Builder** – data privacy and censorship resistance is their #1 non-negotiable requirement (developer personas: Privacy-First Builder, Decision Driver #1)

Proof point: OpenAI content filtering: strict. Anthropic content filtering: strict. Gonka content filtering: none (open-weight model, decentralized infrastructure, no content policy team). Verified: prompts that trigger refusals on OpenAI/Anthropic complete without filtering on Gonka (developer personas: Privacy-First Builder, Activation stage).

Honesty note: Current privacy guarantee is architectural (no central log aggregation), not cryptographic (TEE-based encrypted inference is not yet built). This distinction must be stated clearly in all privacy-related messaging. Overclaiming damages credibility irreparably in security communities. (PITFALLS.md: Pitfall 1; developer personas: Privacy-First Builder, Objection #1)

VP5: Inference Gets Cheaper as the Network Grows

Headline: More GPU hosts join, competition drives prices down – the opposite of centralized provider pricing.

What this means for developers: Traditional inference providers increase margins as they scale. Gonka's decentralized network creates the opposite dynamic: as more GPU hosts join (attracted by GNK mining rewards), they compete for inference requests, driving per-token costs down. Sprint Consensus ensures 98% of GPU power serves actual inference requests – near-zero waste compared to proof-of-work chains that burn 100% of compute on hash puzzles (ARCHITECTURE.md: architecture-to-message mapping, Sprint Consensus).

Supporting evidence: - 98% productive compute via Sprint Consensus (ARCHITECTURE.md: USP #3) - Network economics create deflationary inference pricing as host count increases (ARCHITECTURE.md: USP #5) - Mining rewards subsidize GPU hosting costs, enabling below-market pricing without unsustainable token inflation (ARCHITECTURE.md: architecture-to-message mapping, GNK mining rewards)

Resonates most with: - **Startup CTO** – watching inference costs grow linearly with agent count; a provider with structurally declining costs aligns with runway management (developer personas: Startup CTO, Decision Driver #2) - **Weekend Builder** – benefits from lower costs over time without negotiating enterprise discounts (developer personas: Weekend Builder, Decision Driver #1)

Proof point: Sprint Consensus dedicates 98% of GPU compute to serving inference requests, with only 2% spent on consensus. Every competing proof-of-work blockchain wastes 100% of compute on hash puzzles that generate zero productive output (ARCHITECTURE.md: architecture-to-message mapping, 98% productive compute).

4. Per-Persona Differentiation

4.1 Weekend Builder

Persona-specific positioning statement: Gonka is the cheapest way to run an always-on OpenClaw agent because server-side sessions eliminate the heartbeat tax that makes your \$20 hobby project cost \$40.

Top 3 messages ranked by cost (primary decision driver):

1. **“Your heartbeats are half your bill. Gonka eliminates them.”** At the Casual tier, heartbeats consume 44% of monthly tokens – 460,800 tokens per day generating zero user-facing value. Gonka’s sessions reduce this to ~92,000 tokens/day. (pricing analysis: Section 3, Casual tier; pricing analysis: Section 5, Gonka hidden cost analysis)
2. **“\$13/month instead of \$47/month for the same agent.”** Gonka Scenario B pricing with session persistence saves 72% versus Together AI at the Casual tier – the difference between a reasonable hobby expense and an uncomfortable recurring charge. (pricing analysis: Section 5, Casual tier monthly cost projections)
3. **“Add Gonka in 90 seconds. No wallet, no tokens, no SDK.”** Three fields in openclaw.json. API key via GitHub OAuth. No credit card for the free tier. No blockchain interactions. (ARCHITECTURE.md: OpenClaw Configuration Pattern; PITFALLS.md: Pitfall 2)

Lead with: Cost savings (specific dollar amounts), ease of setup (90 seconds, 3 fields), heartbeat waste elimination

Never lead with: Decentralization, GNK tokens, network economics, Sprint Consensus, mining rewards

Elevator pitch (30 seconds): > “You know how your OpenClaw agent costs way more than it should? Half your token bill is heartbeats – your agent resending context every 30 minutes. Gonka keeps that context server-side, so heartbeats cost almost nothing. Same agent, same code, one config change – your bill drops from \$40 to \$13. It takes 90 seconds to set up.”

4.2 Startup CTO

Persona-specific positioning statement: Gonka gives your team production-grade agent infrastructure with built-in session persistence, automatic model tiering, and webhook notifications – the middleware your engineers are currently building from scratch.

Top 3 messages ranked by reliability (primary decision driver):

1. **“Sessions, tiering, and webhooks – built in, not bolted on.”** Your team currently maintains custom middleware for model routing and session management. Gonka provides all three natively: `/v1/sessions` for state persistence, `X-Gonka-Tier` for automatic model routing, and `/v1/webhooks` for event-driven agent architecture. No other provider offers all three. (competitive matrix: Agent Sessions – WIN, Model Tiering – WIN; developer personas: Startup CTO, Pain Point #2)
2. **“73% lower inference costs at the Active tier.”** Six channel-agents on DeepInfra: \$218/month. Same workload on Gonka Scenario B: \$59/month. The savings come from eliminating heartbeat context re-sending via server-side sessions – a structural advantage that scales super-linearly as you add agents. (pricing analysis: Section 6, Active tier monthly cost projections)

3. **“Your staging test will prove it – one agent, one day, real numbers.”** Deploy one agent on staging with Gonka as provider. Measure p95 latency, daily token consumption, and heartbeat cost. Compare to your current provider. The cost difference will be visible in 24 hours. (developer personas: Startup CTO, Activation stage)

Lead with: Agent-native features (sessions, tiering, webhooks), engineering time savings, quantified cost reduction with specific tier data **Never lead with:** Decentralization philosophy, token economics, blockchain architecture, “unstoppable network” language

Elevator pitch (30 seconds): > “Your agents are spending 51% of their tokens on heartbeats, and your team built custom middleware for model routing. Gonka has session persistence, automatic model tiering, and webhooks built into the API – no custom code needed. We tested one of your staging agents and the daily cost dropped from \$7 to \$2. Same agent, same code, different provider URL.”

4.3 Privacy-First Builder

Persona-specific positioning statement: Gonka is the only inference API where an open-weight model runs on decentralized infrastructure with no content policy, no centralized prompt logging, and no authority that can impose filtering after the fact.

Top 3 messages ranked by privacy/censorship resistance (primary decision driver):

1. **“No content filter. No refusals. Your prompt, your responsibility.”** K2.5 is open-weight with no built-in content filtering. Prompts that trigger refusals on OpenAI and Anthropic – legal adversarial arguments, security analyses, medical discussions – complete without interference on Gonka. (developer personas: Privacy-First Builder, Pain Point #1; competitive matrix: content filtering comparison)
2. **“Decentralized means no central prompt log.”** There is no single Gonka entity that aggregates all prompts and responses. No content policy team reviews your agent’s conversations. No terms-of-service update can retroactively restrict what your agent can process. The infrastructure code is open source at github.com/mitgor/gonka-ai-infrastructure – you can audit it yourself. (developer personas: Privacy-First Builder, Gonka Value Proposition; PITFALLS.md: Pitfall 1)
3. **“Agent features that self-hosting cannot match.”** Self-hosted vLLM gives you privacy but not session persistence, automatic tiering, memory API, or webhook notifications. Gonka gives you both: the privacy posture of decentralized open-weight inference AND the agent-native features of a managed platform. (developer personas: Privacy-First Builder, Gonka Value Proposition #2; provider landscape map: Gonka uniquely combines decentralized + agent-native)

Lead with: No content filtering (specific examples of blocked use cases), open-source infrastructure (auditable), practical privacy guarantees (what is real today) **Never lead with:** Decentralization as philosophy, Web3 ideology, blockchain, “trustless” or “permissionless” language, privacy claims beyond what is currently implemented

Elevator pitch (30 seconds): > “You are running sensitive workloads on self-hosted vLLM because OpenAI and Anthropic block your prompts. Gonka serves K2.5 – an open-weight model with no content filtering – on decentralized infrastructure with no central prompt logging. You get the privacy of self-hosting plus session persistence, automatic tiering, and webhooks that vLLM does not have. The infrastructure code is open source – audit it yourself.”

5. Architecture-to-Message Mapping

Every Gonka technical feature mapped to a developer benefit statement, proof point, and competitive context.

Technical Feature	What It Does (Internal)	Developer Benefit Statement	Proof Point	Competitive Context
Decentralized GPU network	Inference runs on distributed GPU hosts incentivized by GNK mining rewards, not centralized data centers	Your inference runs on a network with no single point of failure and no single authority controlling access or content	98% of GPU compute serves inference requests via Sprint Consensus; near-zero waste (ARCHITECTURE.md: USP #3)	OpenRouter: centralized routing proxy. Together AI: own GPU clusters (centralized). OpenAI/Anthropic: centralized with content policies. Akash: decentralized but no agent features (provider landscape map: segment comparison)
GNK mining rewards	GPU hosts earn GNK tokens for serving inference, subsidizing their compute costs and driving competition	Inference costs decrease as more GPU hosts join the network and compete for your requests	Mining rewards create supply-side competition; more hosts = lower per-token cost over time (ARCHITECTURE.md: USP #5)	OpenRouter: 5.5% credit markup with no mechanism for cost reduction. Together AI: fixed pricing on own clusters. Neither has a supply-side incentive mechanism (competitive matrix: Pricing Model)
98% productive compute	Sprint Consensus uses 98% of GPU power for useful inference, 2% for consensus verification	Every GPU cycle serves your requests. No compute wasted on hash puzzles or proof-of-work busywork	98% productive vs 0% productive on traditional PoW chains; directly translates to lower cost per token (ARCHITECTURE.md: architecture-to-message mapping)	No centralized competitor wastes compute on consensus, but no decentralized competitor matches 98% productivity. Akash/Render use separate consensus mechanisms that do not contribute to inference (provider landscape map: decentralized segment)
OpenAI-compatible API	Drop-in replacement for OpenAI /v1/chat/completions endpoint with identical request/response format	Change one URL in your openclaw.json. Everything else – streaming, tool calling, function format – just works	Verified compatible with OpenClaw, CrewAI, and LangGraph integration test suites; K2.5 stable across 200-300 sequential tool calls (competitive matrix: Section 3, Tool Calling)	OpenRouter: also OpenAI-compatible. Together AI: also OpenAI-compatible. Akash: raw GPU compute, no OpenAI-compatible API without manual vLLM setup (provider landscape map: Akash section)
X-Gonka-Session-ID	Server-side conversation persistence via /v1/sessions API; state maintained across requests without client re-sending context	Your agent remembers context without paying for it twice. Heartbeats send only new data, not the full conversation history every 30 minutes	80% reduction in heartbeat token overhead; Casual tier drops from 31.5M to ~12.6M tokens/month (pricing analysis: Section 5, Gonka hidden cost analysis)	OpenRouter: stateless passthrough, LOSE. OpenAI: Assistants API provides threads but requires different API surface, TIE. Anthropic: no session management, LOSE. Together AI: no session management, LOSE (competitive matrix: Section 1, Agent Sessions)
X-Gonka-Tier header	3-tier auto-routing (lite/mid/full K2.5 quantization); agents select cost-performance tier per request via header or automatic content analysis	Classification on the cheap model, reasoning on the strong one – automatically, with no routing code to maintain	3 quantization tiers of K2.5 with automatic content-aware routing; eliminates custom routing middleware (competitive matrix: Section 2, Model Tiering)	OpenRouter: manual model selection from 500+ catalog, LOSE. OpenAI: manual model selection, LOSE. Anthropic: manual model selection, LOSE. Together AI: manual model selection, LOSE. No competitor offers automatic tier routing (competitive matrix: Section 2)
Memory API	Persistent key-value memory store at /v1/memory with TF-IDF search for retrieval across sessions	Give your agent a permanent memory that persists across conversations and survives restarts	Functional keyword-based retrieval via TF-IDF; combined with sessions provides both short-term (session) and long-term (memory) context (competitive matrix: Section 5, Memory / Context Management)	OpenRouter: no memory, LOSE. Together AI: no memory, LOSE. OpenAI: prompt caching reduces cost but does not store persistent memory, WIN on caching. Anthropic: strongest caching economics (90% discount) but no persistent memory, WIN on caching (competitive matrix: Section 5)
Webhook notifications	Push notifications for async task completion, cost threshold alerts, and model updates via /v1/webhooks	Stop polling. Get notified when async tasks complete, costs exceed thresholds, or models update	Event-driven architecture eliminates polling loops; no other compared provider offers webhook notifications (developer personas: Startup CTO, Gonka Value Proposition #3)	No compared provider offers inference-layer webhook notifications. Developers currently implement their own polling or callback mechanisms (ARCHITECTURE.md: architecture-to-message mapping)

Technical Feature	What It Does (Internal)	Developer Benefit Statement	Proof Point	Competitive Context
Multi-model routing	Route requests to different vLLM backends based on model parameter in the request	One API key, one endpoint – specify the model and Gonka routes to the right backend	Multi-model routing built in v1.2 infrastructure; currently limited to K2.5 quantization tiers (ARCHITECTURE.md: Feature Comparison Matrix)	OpenRouter: multi-model routing is their core product (500+ models), WIN on breadth. Together AI: 200+ models available. Gonka's routing is architecturally sound but currently limited to one model family (competitive matrix: Section 8, Model Breadth – Gonka LOSE)
Kimi K2.5	1T parameter MoE model, 76.8% SWE-Bench Verified, native tool calling, Agent Swarm, 131K context window	The best open-source agent model – purpose-built for tool calling and multi-step reasoning at a fraction of frontier model cost	76.8% SWE-Bench Verified; stable across 200-300 sequential tool calls; 131K context window; native Agent Swarm support (competitive matrix: Section 3, verified against v1.2 tests)	GPT-4o: stronger tool calling with structured outputs but 5-8x more expensive per token. Claude Opus 4: premium reasoning at 30x K2.5 cost. Together AI serves K2.5 at \$0.50/\$2.50 – Gonka must beat this on price or differentiate on features (pricing analysis: Section 4)
vLLM serving	Industry-standard inference serving framework with continuous batching, PagedAttention, and optimized throughput	Production-grade inference infrastructure, not a weekend project. The same serving stack that Together AI and DeepInfra use	vLLM is the industry standard for open-model serving; continuous batching optimizes GPU utilization; PagedAttention reduces memory waste (ARCHITECTURE.md: architecture-to-message mapping)	Together AI: also uses optimized serving on own H100/H200/B200 clusters. DeepInfra: also uses optimized serving. Gonka matches on serving quality but differentiates on infrastructure model (decentralized) and agent features (provider landscape map: dedicated inference segment)

6. Vocabulary Guidelines

6.1 Never-Say List

These terms must never appear in developer-facing content (landing pages, documentation, blog posts, API reference, error messages, community replies, social media). They trigger the “is this a crypto thing?” reaction that causes immediate tab-close among Web2 developers (PITFALLS.md: Pitfall 7).

Internal Term	Why It Alienates	Developer-Facing Alternative
Staking	Implies financial risk, lock-up periods, and crypto speculation. Developers associate staking with DeFi protocols and rug pulls.	“Provider requirements” or “compute commitment” (for host-facing docs); omit entirely in developer-facing content
Epochs	Meaningless to Web2 developers. Signals blockchain infrastructure that they do not want to understand.	“Cycles” or “intervals” if timing context is needed; prefer specific durations (“every 4 years”)
Validators	Implies a blockchain consensus mechanism requiring technical understanding of distributed systems.	“Network nodes” or “compute providers” or simply “GPU hosts”
Gas	Immediately signals Ethereum-style transaction fees. Developers will assume they need ETH or tokens to use the API.	“API usage” or “compute cost” – never suggest per-transaction blockchain fees
Slashing	Implies punitive loss of staked tokens. Sounds adversarial and risk-laden.	“Quality enforcement” or “performance standards”
Governance proposals	Signals DAO-style voting mechanisms that developers do not want to participate in to use an inference API.	“Feature requests” or “community feedback”
DePIN	Crypto-insider acronym (Decentralized Physical Infrastructure Networks). Meaningless and alienating to target audience.	“Distributed compute network” or “decentralized infrastructure” (but only in technical context, never as a headline)
Web3	Loaded term associated with crypto hype, NFTs, and speculative token projects. Triggers skepticism in mainstream developers.	Do not use. Describe specific capabilities instead: “open infrastructure,” “community-operated compute”
Tokenomics	Insider term for token economic design. Signals crypto project, not API provider.	“Network economics” or “pricing model” – developers want to know what things cost, not how your token works
Consensus mechanism	Technical blockchain term that implies developers need to understand distributed consensus to use the API.	“How the network verifies quality” – only in an optional technical deep-dive, never in onboarding or marketing
Mining	Signals cryptocurrency mining with its associations of energy waste, GPU hoarding, and environmental harm.	“Hosting” or “serving inference” – GPU hosts serve inference requests, they do not “mine” tokens

Internal Term	Why It Alienates	Developer-Facing Alternative
On-chain	Signals blockchain transactions. Developers will assume they need a wallet and will encounter transaction fees.	“Verified” or “recorded” if provenance matters; omit entirely when describing developer-facing features
Wallet	Single most alienating word for Web2 developers. If this word appears before “API key” on any page, you have lost the developer.	“Account” – developers have accounts, not wallets. Wallet should only appear in GPU host documentation, never developer-facing
Token (as cryptocurrency)	Ambiguous and loaded. In developer context, “token” means LLM tokens. Using it for GNK currency creates confusion and crypto association.	“Credits” for API usage currency. “GNK” when specifically discussing the network token in investor/host-facing materials only
Smart contract	Signals Ethereum/Solidity development. Developers will think they need to deploy contracts to use inference.	Omit from developer-facing content. If needed in technical architecture docs: “automated agreement” or “network rule”
Decentralized (as headline)	Not inherently bad, but leading with it signals a crypto project rather than an API provider. Use as supporting context, not headline.	Use in supporting position: “... powered by decentralized infrastructure” not “Decentralized AI inference platform” (PITFALLS.md: Pitfall 1)

6.2 Approved Vocabulary

Every crypto/blockchain concept that might appear in developer-facing content, mapped to its approved alternative.

Concept	Approved Term	Usage Example
The Gonka network	“Gonka” or “the Gonka network”	“Deploy on Gonka” – not “Deploy on the Gonka decentralized network”
GNK token (in dev context)	“Compute credits” or “API credits”	“Earn credits toward inference costs” – not “Earn GNK tokens”
GNK token (in host context)	“GNK rewards”	“GPU hosts earn GNK rewards for serving inference” (host docs only)
GPU hosts earning rewards	“GPU providers” or “compute providers”	“Thousands of GPU providers compete to serve your requests”
Sprint Consensus	“Quality verification”	“Every response is quality-verified before delivery” (simple)
Mining/proof-of-work	“Serving inference”	“GPU providers serve inference requests and earn rewards”
Node operators	“GPU hosts” or “compute providers”	“Our network of GPU hosts provides redundant infrastructure”
Token emission schedule	“Network growth incentives”	“Incentives attract more GPU providers, driving costs down”
Proof of useful work	“Productive compute”	“98% of GPU power serves your requests”
Decentralized infrastructure	“Distributed infrastructure” or “redundant infrastructure”	“Gonka runs on distributed infrastructure across thousands of GPU hosts”
Permissionless	“Open access” or “no restrictions”	“Open access inference with no content filtering”
Trustless	“Independently verifiable”	“Infrastructure code is open source and independently verifiable”
Block rewards	“Host earnings”	“GPU hosts earn competitive rewards for providing compute”
Network fees	“Usage-based pricing”	“Pay only for what you use – standard per-token pricing”
DAO / governance	“Community input” or “feedback”	“Share feedback to shape the product roadmap”
Mainnet / testnet	“Production” / “staging”	“Gonka’s production network” – not “Gonka mainnet”
Gas fees	(never reference)	Omit. There are no per-transaction blockchain fees for developers
Wallet connection	“API key”	“Get your API key in 60 seconds via GitHub OAuth”
On-chain verification	“Automated quality checks”	“Every inference response passes automated quality checks”

6.3 Context Rules

Crypto terminology is acceptable in specific, clearly delineated contexts where the audience expects it:

Acceptable contexts:

1. **“How It Works Under the Hood” documentation section.** A clearly labeled optional deep-dive for technically curious developers. Gate this behind a click (“Want to understand the network architecture?”) so developers never encounter it during onboarding or standard API usage. Acceptable terms: consensus mechanism, Sprint Consensus, mining rewards, GNK token, proof of useful work.
2. **Investor-facing materials.** Pitch decks, token economics documents, and materials for crypto-native investors expect and require blockchain terminology. Use full crypto vocabulary. These materials must never be linked from developer-facing pages.

3. **GPU host documentation.** Documentation for GPU providers who join the network to earn GNK rewards. These users are crypto-aware (they are running nodes for token incentives) and expect terms like staking, rewards, slashing penalties, and epochs. Host docs should be a separate documentation section from developer API docs.
4. **Conference talks at crypto/Web3 events.** When presenting at Token2049, ETHDenver, or similar events, use the audience’s vocabulary. But even here, lead with the developer use case first, then explain the infrastructure.
5. **Internal communications.** Team Slack, internal docs, planning documents – use whatever terminology is efficient for internal understanding.

Never acceptable:

- Landing page (docs.gonka.ai homepage)
- API reference documentation
- Quickstart guide
- Pricing page
- Error messages returned by the API
- Social media posts targeting developers (Twitter/X, Reddit, Hacker News)
- Blog posts about developer features
- OpenClaw integration documentation
- Community replies in OpenClaw Discord or GitHub

7. Competitive Differentiation Statements

Per-persona competitive positioning against the single competitor each persona is most likely comparing Gonka to (ARCHITECTURE.md: Anti-Pattern 3, focus on 2-3 per persona, not all competitors).

7.1 Weekend Builder: Gonka vs OpenRouter

The competitor context: OpenRouter is the default inference provider in OpenClaw – zero configuration required. It is the path of least resistance. The Weekend Builder has never changed their provider and sees no reason to unless the cost savings are dramatic and the switch is trivially easy. (competitive matrix: Key Takeaway #3; developer personas: Weekend Builder, Objection #2)

Differentiation statement:

Unlike OpenRouter which charges for every token of repeated context plus a 5.5% credit markup, Gonka maintains your agent’s conversation server-side so heartbeats cost almost nothing – dropping your monthly bill from ~\$52 to ~\$13 for the same agent with one config change. (pricing analysis: Sections 3 and 5; competitive matrix: Agent Sessions, OpenRouter scored LOSE)

Supporting claims:

Dimension	OpenRouter	Gonka	Verdict
Agent Sessions	Stateless passthrough; all session management client-side	Server-side session persistence via /v1/sessions API	Gonka WIN (competitive matrix)
Pricing	Per-token passthrough + 5.5% credit markup (\$52/month Casual tier after markup)	Per-token, no markup (Scenario B: ~\$13/month Casual tier with sessions)	Gonka advantage (pricing analysis)
Model Breadth	500+ models from all providers	1 model (K2.5) in 3 tiers	OpenRouter WIN (competitive matrix)
Setup	Built-in to OpenClaw, zero config	Custom provider, 3 fields in openclaw.json	OpenRouter advantage (developer personas)

Honest concession: OpenRouter offers 500+ models and zero-config OpenClaw integration. Gonka’s single-model offering and manual configuration are real disadvantages. The cost savings must be compelling enough to justify the switch. (competitive matrix: Model Breadth – Gonka LOSE)

7.2 Startup CTO: Gonka vs Together AI

The competitor context: Together AI is the performance and cost pick for teams running K2.5 at scale. Own GPU clusters, no middleman markup, lowest verified K2.5 pricing (\$0.50/\$2.50 per 1M tokens). The Startup CTO is either using Together AI already or evaluating it against direct OpenAI API access. (pricing analysis: Section 4; competitive matrix: Pricing Model, Together AI scored WIN)

Differentiation statement:

Unlike Together AI which offers raw per-token inference with no agent-specific features, Gonka provides server-side session persistence, automatic model tiering, and webhook notifications built into the API – eliminating the custom middleware your team currently maintains and reducing Active tier costs from \$218/month to \$59/month through session-based heartbeat savings. (competitive matrix: Agent Sessions and Model Tiering, Together AI scored LOSE on both; pricing analysis: Section 6; developer personas: Startup CTO, Pain Points)

Supporting claims:

Dimension	Together AI	Gonka	Verdict
Agent Sessions	No session management; stateless API	Server-side session persistence	Gonka WIN (competitive matrix)
Model Tiering	Manual model selection; no auto-routing	3-tier auto-routing via X-Gonka-Tier header	Gonka WIN (competitive matrix)
Per-token Pricing	\$0.50/\$2.50 for K2.5 (lowest verified)	TBD (Scenario B: \$0.35/\$1.75)	Together AI advantage until Gonka pricing is published (pricing analysis)
Uptime / Reliability	Reliable track record, no formal SLA	No SLA, unproven at scale	Together AI advantage (competitive matrix: Uptime – Gonka LOSE)

Honest concession: Together AI has the lowest verified K2.5 pricing and a reliable operational track record. Gonka’s pricing is not yet published, and its production reliability is unproven. The agent-native feature advantage must be demonstrated on the CTO’s staging environment before trust is established. (competitive matrix: Uptime – Gonka LOSE; pricing analysis: Section 4, Gonka pricing TBD)

7.3 Privacy-First Builder: Gonka vs Akash Network

The competitor context: Akash Network is the closest decentralized alternative – a Kubernetes-as-a-Service platform for GPU compute with 65+ datacenters and a reverse auction pricing model. AkashML launched an OpenAI-compatible managed inference API in November 2025. The Privacy-First Builder evaluating decentralized options will consider both. (provider landscape map: Akash/AkashML section)

Differentiation statement:

Unlike Akash Network which provides raw GPU compute requiring manual vLLM deployment and configuration, Gonka offers a managed OpenAI-compatible inference API with agent-native extensions – session persistence, memory API, automatic tiering, and webhooks – that self-hosted vLLM cannot match, while maintaining the decentralized infrastructure and content freedom that make both platforms appealing to privacy-conscious developers. (provider landscape map: Gonka vs Akash comparison; ARCHITECTURE.md: Feature Comparison Matrix)

Supporting claims:

Dimension	Akash Network	Gonka	Verdict
API Compatibility	AkashML: OpenAI-compatible (since Nov 2025); raw Akash: manual vLLM setup	OpenAI-compatible API, drop-in for OpenClaw	Both now offer compatible APIs; Gonka’s has been tested with OpenClaw/CrewAI/LangGraph (ARCHITECTURE.md)
Agent Features	No session management, no memory API, no tiering, no webhooks	Full agent-native feature set (sessions, memory, tiering, webhooks)	Gonka advantage (provider landscape map: Akash has agent-awareness level “None”)
Content Filtering	No content policy (decentralized)	No content policy (decentralized, open-weight K2.5)	Equivalent privacy posture (both decentralized, both censorship-resistant)

Dimension	Akash Network	Gonka	Verdict
Privacy Guarantees	Distributed compute, no central logging	Distributed compute, no central logging; neither offers TEE yet	Equivalent current state; both need TEE for cryptographic guarantees (developer personas: Privacy-First Builder, Objection #1)
Model Selection	Multiple models via AkashML managed inference	Single model (K2.5) in 3 quantization tiers	Akash advantage on breadth (provider landscape map)

Honest concession: Akash has a longer operational track record as a decentralized compute platform and now offers managed inference via AkashML with multiple model options. Neither platform currently offers TEE-based encrypted inference – the privacy guarantees are architectural (no central logging), not cryptographic. For the highest-sensitivity workloads, both require the same trust assumptions. (developer personas: Privacy-First Builder, Objection #1; provider landscape map: Akash section)

8. Objection Handling Quick Reference

Consolidated objection responses for rapid reference during community interactions, sales conversations, and content creation. Each response is grounded in research evidence.

Objection	Persona(s)	Response	Evidence
“I have never heard of Gonka.”	All	Lead with the developer pain point (heartbeat costs, content filtering) not the brand. “You know how half your OpenClaw bill is heartbeats? There is a provider that eliminates that.” Let the value proposition create the introduction.	(ARCHITECTURE.md: Objection Map; developer personas: universal blocker #1)
“Is this a crypto thing?”	All	“No. It is an API. You get an API key, paste a URL into your openclaw.json, and make inference calls. The infrastructure happens to be decentralized, which is why it is cheaper, but you never interact with blockchain, wallets, or tokens.”	(PITFALLS.md: Pitfalls 1, 2, 7; developer personas: universal blocker #2)
“OpenRouter already works, why switch?”	Weekend Builder	“It works, but you are paying a 5.5% credit markup plus full token cost on every heartbeat. Gonka keeps your conversation server-side – heartbeats send only new data. That drops your bill from ~\$52 to ~\$13.”	(competitive matrix: Agent Sessions; pricing analysis: Sections 3, 5)
“Decentralized = unreliable.”	Startup CTO	“That is a fair concern. Test on staging first. Deploy one agent, measure p95 latency and uptime for a week, compare to your current provider. We want you to evaluate with data, not promises.”	(competitive matrix: Uptime – Gonka LOSE acknowledged; developer personas: Startup CTO, Objection #1)
“K2.5 is not Claude or GPT.”	Weekend Builder, Startup CTO	“K2.5 scores 76.8% on SWE-Bench Verified and is stable across 200-300 sequential tool calls. For agent workloads – tool calling, multi-step reasoning, code generation – it is competitive with frontier models at 5-10x lower cost.”	(competitive matrix: Section 3, Tool Calling; ARCHITECTURE.md: K2.5 flagship)
“We need multiple models.”	Startup CTO	“Fair point – Gonka currently serves K2.5 in 3 quantization tiers. For teams needing model diversity, run Gonka alongside your existing provider: use Gonka for K2.5 workloads (where sessions save you 73%), keep OpenAI/Anthropic for tasks that need their proprietary models.”	(competitive matrix: Model Breadth – Gonka LOSE acknowledged; pragmatic dual-provider approach)
“How do I know you do not log prompts?”	Privacy-First Builder	“The infrastructure code is open source at github.com/mitgor/gonka-ai-infrastructure – audit it. No central entity aggregates prompts. We are honest that this is architectural privacy (no central log aggregation), not cryptographic privacy (TEE-based encryption is on the roadmap, not shipped). We will not overclaim.”	(developer personas: Privacy-First Builder, Objection #1; PITFALLS.md: Pitfall 1)

Objection	Persona(s)	Response	Evidence
"Can it handle production scale?"	Startup CTO	"We have integration test results for OpenClaw, CrewAI, and LangGraph. But you should not trust test results – deploy on staging, run your workload for a week, and measure. Production confidence comes from your data, not ours."	(developer personas: Startup CTO, Objection #3; ARCHITECTURE.md: Objection Map)
"I do not want to explain crypto to my investors."	Startup CTO	"Neither do we. Gonka's developer-facing materials never mention blockchain, tokens, or mining. Share our API docs, not our architecture paper. Your investors see 'inference provider with 73% cost savings' not 'decentralized GPU network.'"	(PITFALLS.md: Pitfall 7; developer personas: Startup CTO, Objection #4)

9. Agent-Native Pitch

Why OpenClaw Agents Themselves Would Prefer Gonka

This section addresses a unique aspect of Gonka's positioning: OpenClaw agents are not just tools that developers configure – they are autonomous systems that make runtime decisions about which inference provider to call, which model to use, and how to optimize their own costs. An agent that can programmatically evaluate providers is a "buyer" in its own right (ARCHITECTURE.md: Anti-Pattern 5).

The agent's decision calculus:

1. **Session persistence = less code.** An agent using Gonka does not need to implement its own context management. It creates a session, appends messages, and the server maintains state. This is simpler agent code with fewer failure modes. (competitive matrix: Agent Sessions – WIN)
2. **Tier selection = smarter resource use.** An agent using the `X-Gonka-Tier` header can make per-request decisions about cost vs quality based on task complexity. Classification gets `lite`, reasoning gets `full` – the agent optimizes its own costs in real time. (competitive matrix: Model Tiering – WIN)
3. **Webhooks = event-driven architecture.** An agent can fire-and-forget long-running tasks and receive a webhook notification when they complete, rather than polling. This is more efficient and reduces wasted inference cycles. (ARCHITECTURE.md: architecture-to-message mapping, Webhook notifications)
4. **Memory API = persistent knowledge.** An agent can store and retrieve long-term facts across sessions without rebuilding context from conversation history. TF-IDF search is functional for keyword-based retrieval today. (competitive matrix: Memory / Context Management – TIE)

The pitch to developers who build agent systems:

"Gonka is not just cheaper inference. It is an API designed for how agents actually work – maintaining state across calls, routing tasks to the right model tier, notifying on completion, and remembering facts across sessions. Your agent writes less code and spends fewer tokens because the infrastructure handles what agents need natively."

10. Message Hierarchy Summary

For quick reference when creating any developer-facing content:

Primary Message (Use First)

"Gonka cuts your OpenClaw agent costs by up to 73% because your agent stops paying for context it already has."

Secondary Messages (Support Primary)

- “Session persistence eliminates heartbeat waste – the #1 hidden cost in agent workloads.”
- “Automatic model tiering puts classification on the cheap model and reasoning on the strong one.”
- “One config change, 90 seconds, no wallet, no tokens, no SDK.”

Tertiary Messages (Use When Relevant to Audience)

- “No content filters, no prompt logs, no permission required.” (Privacy-First only)
- “Inference gets cheaper as the network grows.” (Cost-conscious audiences)
- “Agent-native features that no other provider offers.” (Technical audiences evaluating agent infrastructure)

Never-Lead-With Messages

- Decentralization, blockchain, Web3, DePIN
- GNK tokens, mining rewards, staking
- Sprint Consensus (unless in technical deep-dive)
- “Unstoppable network” or “permissionless” language
- Price comparisons without published Gonka pricing (all cost claims must note Gonka pricing is TBD/scenario-based)

Document: gonka_message_house.md / Version 1.0 / 2026-04-01 Feeds into: Phase 18 (Channel Strategy), Phase 19 (Partnership & Ecosystem) Sources: gonka_competitive_feature_matrix.md, gonka_developer_personas.md, gonka_agent_pricing_analysis.md, gonka_provider_landscape_map.md, ARCHITECTURE.md, PITFALLS.md